

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)**Department of Computer Science and Engineering****ASSESSMENT TOOL 1 – Constraint-Based Problem Solving**

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO5– Naturalization (BL5,BL6)	Assessment:	Assessment Tool 1 – Constraint Based Problem Solving
Weightage:	30% of CIA	Total Marks:	100
CO5 Statement:	<i>Analyze computational problems using backtracking techniques and complexity classes such as P, NP, NP-Hard, and NP-Complete for combinatorial problem solving.</i>		
Student Name:	CH.Bhavya Sri	Reg. No.:	192411249
Section / Batch:	Slot C	Date:	25.08.2026

Instructions to Students

- Answer the following question. Total Marks: 100.
- Apply backtracking techniques systematically for combinatorial problem solving.
- Clearly classify problems under P, NP, NP-Hard, and NP-Complete categories wherever required.
- Provide proper justification, state-space representation, and complexity analysis for all solutions.
- Neat presentation and logical explanation are mandatory.

Question Type	Focus Area	Questions
Constraint Based Problem Solving	Analyze combinatorial problems using backtracking and complexity classes such as P, NP, NP-Hard, and NP-Complete.	Q1

SECTION A – CONSTRAINT-BASED PROBLEM SOLVING

A company must assign jobs to employees such that each job is completed within a deadline and no employee handles more than one job at a time. Clearly define the problem and identify constraints related to time, resource availability, and job requirements. Analyze how these constraints interact and prioritize them logically. Develop a feasible solution using backtracking or constraint satisfaction techniques. Apply relevant concepts such as pruning and recursive assignment. Justify how your solution satisfies all constraints efficiently.

Code of Conduct

I CH.Bhavya Sri/192411249 _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: CH.Bhavya Sri _____

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	25%				
Constraint Analysis	25%				
Solution Development	25%				
Application of Concepts	15%				
Justification	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name: _____	Name: _____	Name: _____
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____



SIMATS Online Programming Lab
DAA PROGRAMMING

 CHENNABOINA BHAVYA SRI
192411249RC

CO5 AT1-Constraint-Based Problem Solving
← Back

Language: Python C C++ Java

QUESTIONS

Q1
Job Scheduling (Backtracking / NP-Hard Problem)
100 marks

0/1 answered

Q1 Job Scheduling (Backtracking / NP-Hard Problem) 100 marks

A company must assign jobs to employees such that each job is completed within a deadline and no employee handles more than one job at a time. Clearly define the problem and identify constraints related to time, resource availability, and job requirements. Analyze how these constraints interact and prioritize them logically. Develop a feasible solution using backtracking or constraint satisfaction techniques. Apply relevant concepts such as pruning and recursive assignment. Justify how your solution satisfies all constraints efficiently.

Your Code

```

1: def scheduling(jobs):
2:     jobs = list of tuples (job_id, deadline, profit)
3:     n = len(jobs)
4:     x_deadline = max(job[1] for job in jobs)
5:
6:     slot array to track which job is assigned to which time slot
7:     slots = [-1] * (max_deadline + 1) # slots[0] unused, 1-indexed
8:
9:     sort jobs by profit descending (greedy ordering improves backtracking)
10:    jobs = sorted(jobs, key=lambda x: x[2], reverse=True)
11:
12:    result = []

```

Input

stdin input

Output

```

Job Assignment (Job, Slot):
J2 -> Slot 1
J1 -> Slot 2
J5 -> Slot 3

```

This question asks us to **schedule different jobs among employees** while satisfying certain conditions.

1. Main problem

We have:

- Several **jobs**
- Several **employees**
- Each job has a **deadline**
- Each job may have specific **requirements**
- An employee can work on **only one job at a time**

Our goal is to assign every job to a suitable employee and time slot without violating any condition.

2. Important constraints

Time constraint:

A job must be completed before or within its deadline.

Employee/resource constraint:

One employee cannot handle two jobs at the same time.

Job requirement constraint:

A job should be assigned only to an employee who has the required skill/resource.

3. Backtracking idea

Backtracking means:

Assign → Check → If valid, continue → If invalid, go back and try another choice.

For example:

Job 1 → Employee 1 → Valid
Job 2 → Employee 1 → Invalid
↓
Try Employee 2
Job 2 → Employee 2 → Valid

If an assignment violates a deadline or employee availability, we **prune** that choice instead of continuing.

4. Recursive assignment

The solution can be represented as:

Schedule Job 1
↓
Check constraints
↓
Schedule Job 2
↓
Check constraints
↓
Schedule Job 3
↓
...
↓
All jobs assigned → Solution found

5. Why pruning is useful

Without pruning, the program tries many unnecessary combinations.

With pruning:

If a partial assignment already violates a constraint, stop exploring that path.

This makes the backtracking solution more efficient.